Aprendizaje de Máquina
Sesión 25: Árboles de decisión y ensambles de árboles

Angel Rodríguez

Avisos:

- Último examen parcial: martes 10 de Noviembre. Guía la subo hoy en la noche.
- Exposición proyectos finales:
  1. jueves 26 de Noviembre en horario y salón de clase (4 equipos).
  2. jueves 3 de Diciembre, de 19:00 a 20:59 horas en el CN218 (4 equipos).
- Fecha de entrega proyecto final: jueves 26 de Noviembre.

Proyecto final: análisis y clasificación de textos

Objetivo

Desarrollar un sistema que **descubra grupos temáticos** en documentos y **clasifique textos nuevos**, justificando las decisiones mediante experimentos reproducibles.

Componentes del proyecto

1. **Representación (20 %):** limpieza, BoW y TF–IDF.
2. **No supervisado (25 %):** K-means, jerárquico, métricas y reducción de dimensionalidad.
3. **Supervisado (25 %):** comparar 3 clasificadores y seleccionar el mejor.
4. **Generalización (20 %):** sobreajuste, regularización, dimensionalidad y curvas de aprendizaje.
5. **Discusión (10 %):** interpretación, limitaciones y conclusiones.

Entregables

- Reporte escrito.
- Código Python reproducible.
- Presentación (12 min) y preguntas (8 min).

Condiciones mínimas

≥3,000 textos, ≥4 categorías, 2 representaciones, 2 agrupamientos, 3 clasificadores y 4 experimentos.

Proyecto final: dinámica general

- El proyecto se realizará en **equipos de dos personas**.
- **Entregables:** cada equipo deberá entregar:
  - Un **reporte en PDF**.
  - El **código Python** en un archivo aparte y documentado.
- **Evaluación:**
  - Reporte en PDF: **30 %** (calificación por equipo).
  - Exposición oral: **70 %** (calificación individual).
- Cada equipo realizará una **exposición oral de 12 minutos** para presentar su proyecto.
- **Participación individual:** se harán **preguntas a cada integrante** para verificar que todos comprendan el proyecto completo.
- **Importante:** se **restarán puntos por respuestas equivocadas**. Todos deben dominar la teoría, el código y los resultados.

Proyecto final

Hoy en la noche subo todos los detalles a una actividad de blackboard.

¿Qué es un árbol de decisión?

- Modelo supervisado que puede resolver problemas de **clasificación** y **regresión**.
- Cada nodo interno contiene una condición, por ejemplo $x_j \leq t$.
- Cada rama representa el resultado de esa condición.
- Cada hoja proporciona una predicción.
- Puede capturar relaciones no lineales sin escalar ni centrar los atributos.

Ejemplo

¿La longitud del pétalo es menor o igual a 2,45 cm? Si sí, predecimos *setosa*; si no, hacemos otra pregunta.

Cómo se realiza una predicción

1. Comenzamos en la **raíz**.
2. Evaluamos la condición del nodo.
3. Si se cumple, avanzamos por la rama izquierda; de lo contrario, por la derecha.
4. Repetimos hasta alcanzar una **hoja**.
5. La hoja determina la predicción.

Ejemplo Iris

- Longitud $\leq$ 2,45: *setosa*.
- Longitud $>$ 2,45 y anchura $\leq$ 1,75: *versicolor*.
- Longitud $>$ 2,45 y anchura $>$ 1,75: *virginica*.

Interpretación de los nodos

- **samples**: número de ejemplos de entrenamiento que llegan al nodo.
- **value**: cantidad de ejemplos de cada clase (o cantidades ponderadas).
- **class**: clase mayoritaria, usada para predecir.
- **gini**: medida de mezcla de clases dentro del nodo.

Pregunta

Si una hoja contiene 49 flores versicolor y 5 virginica, ¿qué clase predice? ¿Con qué confianza empírica?

Impureza de Gini

Para un nodo i con K clases, definimos

$$\boxed{G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2}$$

donde $p_{i,k}$ es la proporción de ejemplos de clase k en el nodo i.

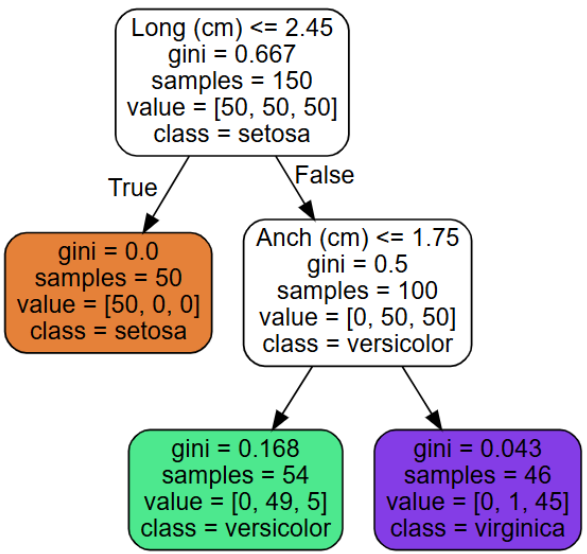
- Si todas las observaciones pertenecen a una clase, $G_i = 0$.
- Cuanto más mezcladas estén las clases, mayor es la impureza.

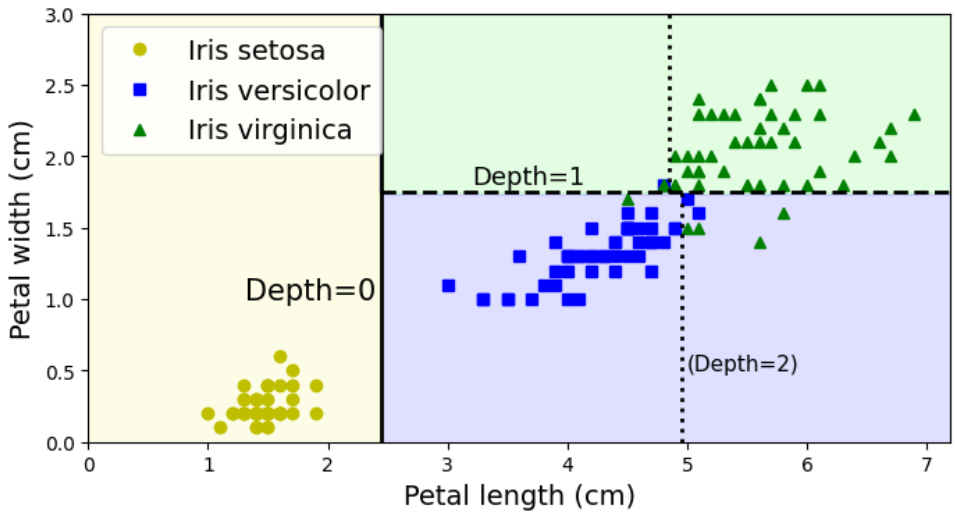
Ejemplo

Para proporciones $(0, 49/54, 5/54)$:

$$G = 1 - (49/54)^2 - (5/54)^2 \approx 0{,}168.$$

Ejemplo


Long (cm) <= 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
Anch (cm) <= 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica


Iris setosa
Iris versicolor
Iris virginica
Depth=1
Depth=0
(Depth=2)
Petal width (cm)
Petal length (cm)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
1
2
3
4
5
6
7

Probabilidades estimadas en las hojas

El árbol estima la probabilidad de clase usando la proporción de ejemplos de esa clase en la hoja:

$$\hat{P}(Y = k \mid X = x) = \frac{m_{hoja,k}}{m_{hoja}},$$

donde:

- m_{hoja}: número total de muestras que llegaron a esa hoja durante el entrenamiento.
- $m_{hoja,k}$: número de esas muestras que pertenecen a la clase k.
- k: clase para la cual queremos estimar la probabilidad.

```
print(arbol.predict_proba([[5, 1.5]]).round(3))
print(arbol.predict([[5, 1.5]]))
```

Resultado esperado: probabilidades $(0, 0{,}907, 0{,}093)$ y clase 1.

Limitación Todos los puntos que llegan a la misma hoja reciben exactamente las mismas probabilidades.

Algoritmo CART: construcción del árbol

CART construye el árbol mediante un procedimiento **recursivo y greedy**.

1. Comienza con todos los ejemplos de entrenamiento en el nodo raíz.
2. Para cada característica k, considera diferentes umbrales t_k.
3. Evalúa la impureza ponderada de los nodos que produciría cada división:

$$J(k, t_k) = \frac{m_{izq}}{m} G_{izq} + \frac{m_{der}}{m} G_{der},$$

   y selecciona la característica y el umbral que minimizan la función de costo:

$$(k^*, t^*) = \arg \min_{k, t_k} J(k, t_k).$$

4. Divide los ejemplos en dos nodos hijos y repite el procedimiento en cada uno.
5. **Criterios de parada:** nodo puro, profundidad máxima, número mínimo de muestras, etc.

Complejidad computacional

Sea m el número de ejemplos y n el número de características.

- En un árbol aproximadamente balanceado, la profundidad es del orden de $\log_2 m$.
- **Predicción**: aproximadamente $O(\log m)$ comparaciones por instancia.
- **Entrenamiento**: aproximadamente $O(nm \log m)$ (por nodo) con búsqueda exhaustiva de las divisiones candidatas en el nodo.
- Limitar profundidad y atributos considerados puede acelerar el entrenamiento.

Entropía como criterio de impureza

Además de Gini, podemos utilizar la **entropía**:

$$H_i = -\sum_{k:p_{i,k}>0} p_{i,k} \log_2 p_{i,k}$$

- $H_i = 0$ cuando el nodo contiene una sola clase.
- Una mezcla equilibrada de clases aumenta la entropía.
- Gini y entropía suelen generar árboles similares.
- Gini suele ser algo más rápido de calcular.
- Cuando difieren, entropía genera árboles más balanceados.

Ejemplo

Para $(49/54, 5/54)$, $H \approx 0{,}445$ bits.

Sobreajuste y regularización

Un árbol sin restricciones puede adaptarse demasiado a los ejemplos de entrenamiento.

- **Árbol profundo**: mayor flexibilidad, pero mayor riesgo de sobreajuste.
- **Árbol poco profundo**: menos varianza, pero puede subajustar.
- Regularizar significa limitar su capacidad de ajustar detalles del conjunto de entrenamiento.

Hiperparámetros de regularización

Hiperparámetro	Efecto
`max_depth`	Limita la profundidad máxima.
`min_samples_split`	Mínimo de muestras para dividir un nodo.
`min_samples_leaf`	Mínimo de muestras en cada hoja.
`max_leaf_nodes`	Limita el número de hojas.
`max_features`	Limita atributos evaluados en cada división.
`min_impurity_decrease`	Exige una mejora mínima de impureza.
`ccp_alpha`	Controla la *poda de mínima complejidad-costo* o MCCP (minimal cost-complexity pruning). Mayor `ccp_alpha`, más poda. Default = 0.

Regla práctica

Aumentar restricciones reduce la complejidad del árbol; demasiada regularización puede causar subajuste.
Reducir `max_depth` y aumentar `min_samples_leaf` suelen ser buenos puntos de partida.

Poda de árboles

Otra forma de regularización es **podar** ramas que aportan poca mejora.

- Se puede crecer un árbol y después eliminar subárboles innecesarios.
- En scikit-learn, `ccp_alpha` controla la poda por complejidad de costo.
- Un valor mayor suele producir un árbol más pequeño.

Poda por complejidad-costo (MCCP)

CART puede construir árboles muy profundos que **sobreajustan** los datos de entrenamiento.

Minimal Cost-Complexity Pruning (MCCP) simplifica el árbol eliminando ramas que aportan poca mejora respecto a su complejidad.

Función objetivo (penaliza el tamaño de árbol)

$$\boxed{R_\alpha(T) = R(T) + \alpha|T|}$$

- $R(T) = \sum_{\ell \in \text{hojas}(T)} \frac{m_\ell}{m} G_\ell$: impureza total ponderada de las hojas.
- $|T|$: número de hojas del árbol.
- $\alpha \geq 0$: penalización por complejidad.

La idea es encontrar un equilibrio entre dos objetivos:

1. Tener poca impureza: $R(T)$.
2. Tener un árbol pequeño: $\alpha|T|$.

MCCP: ¿qué ramas se eliminan?

MCCP utiliza una estrategia llamada **weakest-link pruning** (poda del eslabón más débil).

1. Se construye inicialmente un árbol grande.
2. Para cada nodo interno, se evalúa el costo de sustituir todo su subárbol por una hoja.
3. Se calcula:

$$\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

4. Se poda el subárbol con menor α_{eff} y con $\alpha_{eff} \leq$ `cpp_alpha`.
5. Se repite el procedimiento para generar árboles progresivamente más pequeños.

Interpretación de α_{eff}

$$\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

- **Numerador:** $R(t) - R(T_t)$

  Mide cuánto aumenta la impureza al sustituir el subárbol por una hoja.

- **Denominador:** $|T_t| - 1$

  Representa el número neto de hojas eliminadas.

Interpretación

α_{eff} mide el **aumento de impureza por cada hoja eliminada**.

- α_{eff} pequeño: podemos simplificar el árbol perdiendo poca calidad de ajuste.
- α_{eff} grande: la rama aporta una reducción importante de impureza en relación con su complejidad.

Árboles de regresión

En regresión, las hojas predicen **valores numéricos** en lugar de clases.

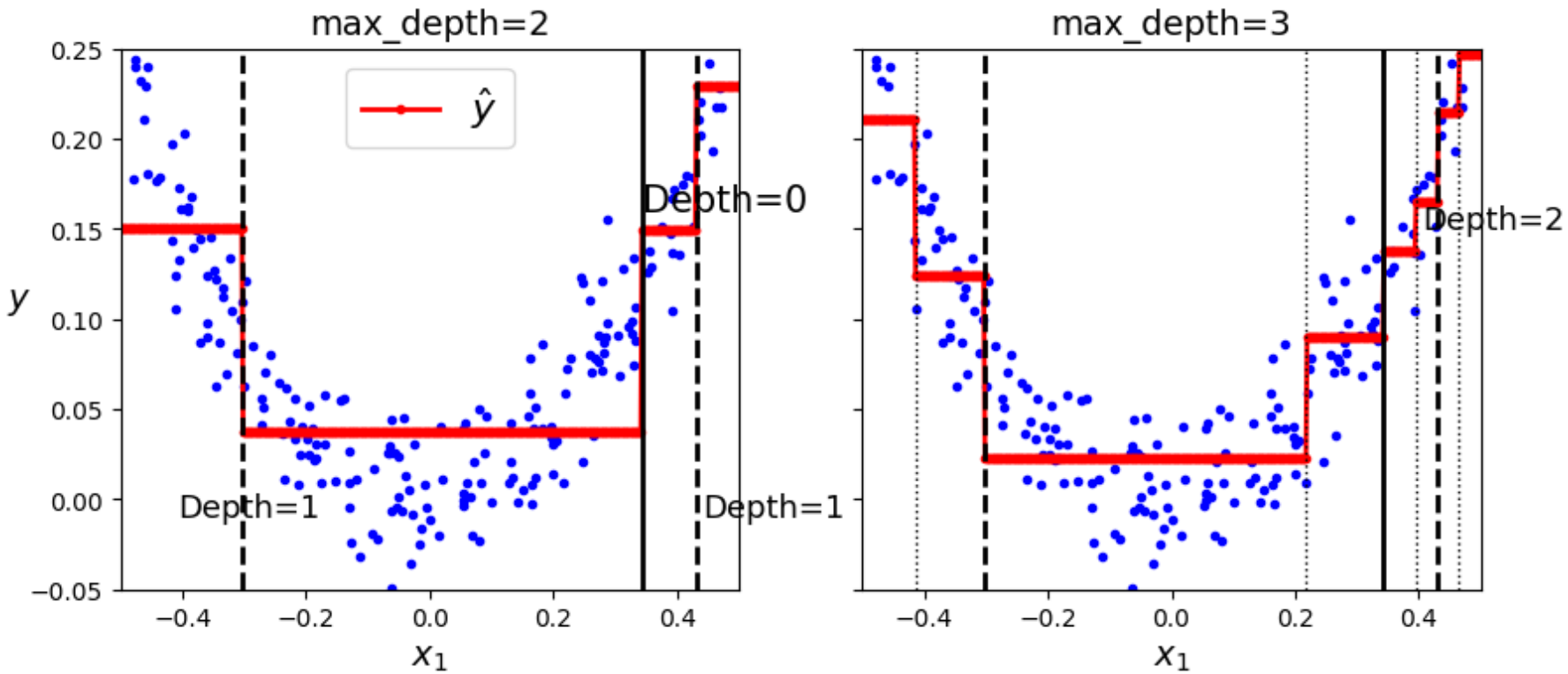
- Cada hoja predice la media de los valores objetivo de los ejemplos que llegan a ella.
- El árbol divide el espacio en regiones con predicciones constantes.
- CART selecciona divisiones que reducen el error cuadrático medio (MSE).

Comparación

Clasificación: pureza de clases. Regresión: dispersión de los valores objetivo.

Observación:

Aunque la relación verdadera es cuadrática, el árbol aproxima la curva mediante escalones:



CART para regresión

El costo de una división se calcula como

$$\boxed{J(k, t_k) = \frac{m_{izq}}{m} MSE_{izq} + \frac{m_{der}}{m} MSE_{der}}$$

con

$$MSE_{nodo} = \frac{1}{m_{nodo}} \sum_{i \in nodo} (y_i - \bar{y}_{nodo})^2,$$

$$\bar{y}_{nodo} = \frac{\sum_{i \in nodo} y_i}{m_{nodo}},$$

Interpretación

Se buscan regiones cuyos valores objetivo estén lo más cerca posible de su media.

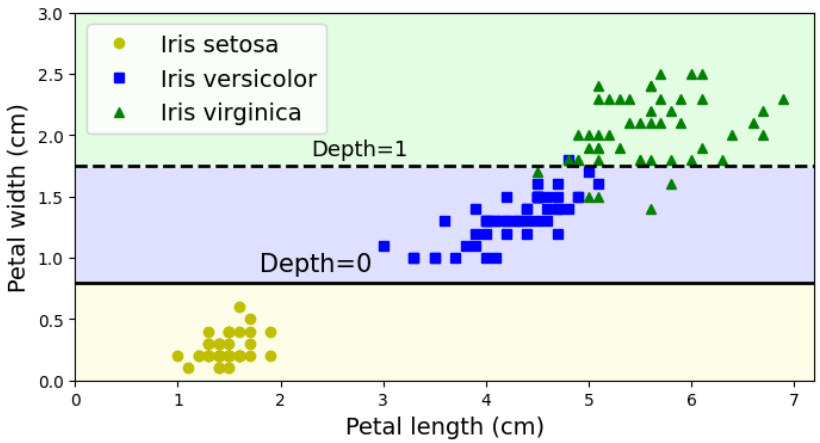
Alta varianza de los árboles

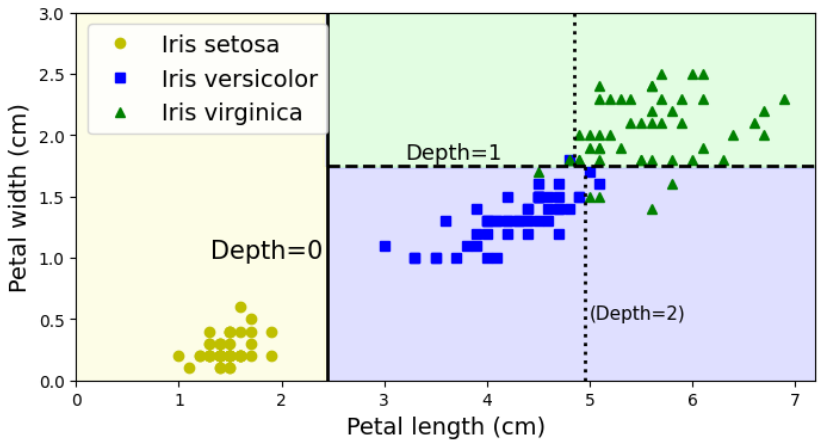
- Pequeños cambios en los datos pueden modificar considerablemente el árbol.
- Cambios en hiperparámetros también pueden producir estructuras distintas.
- Fijar `random_state` facilita reproducir los resultados, pero no elimina la sensibilidad del modelo.
- Combinar muchos árboles permite reducir la varianza: idea central de los **bosques aleatorios**.

Conclusión

Los árboles son interpretables y flexibles, pero suelen requerir regularización y una evaluación cuidadosa de generalización.

Ejemplo Sensibilidad:





Ensemble Learning: idea general

Aprendizaje por ensambles

Un **ensamble** combina las predicciones de varios modelos para construir un predictor que, frecuentemente, generaliza mejor que sus componentes individuales.

La idea está relacionada con la *sabiduría colectiva*: la combinación de distintas opiniones puede producir una respuesta más precisa que una opinión individual.

- Los modelos individuales se denominan **predictores base**.
- Pueden utilizar diferentes algoritmos o diferentes subconjuntos de entrenamiento.
- Sus predicciones se combinan mediante votación, promedios o un modelo adicional.

Idea fundamental

Un ensamble funciona especialmente bien cuando sus predictores son suficientemente precisos y cometen errores diferentes.

Principales métodos de ensamble

1. **Voting:** combina las predicciones de distintos clasificadores mediante votación.
2. **Bagging y Pasting:** entrena múltiples modelos utilizando diferentes subconjuntos de datos.
3. **Random Forests:** combina árboles de decisión incorporando aleatoriedad en los datos y las características.
4. **Boosting:** construye modelos secuencialmente, intentando corregir los errores anteriores.
5. **Stacking:** utiliza un modelo adicional para aprender cómo combinar las predicciones.

Objetivo

Mejorar la capacidad de generalización, frecuentemente reduciendo la varianza o el sesgo del predictor final.

Voting Classifiers: votación mayoritaria

Supongamos que entrenamos M clasificadores $f_1, \ldots, f_M$ sobre un problema de clasificación.
Cada clasificador produce una predicción $\hat{y}_j(x)$ para una observación x.

Hard Voting

La predicción del ensamble es la clase que recibe más votos:

$$\hat{y}(x) = \arg\max_k \sum_{j=1}^{M} \mathbb{I}\big(\hat{y}_j(x) = k\big)$$

donde $\mathbb{I}(\cdot)$ es la función indicadora.

Por ejemplo, si tres clasificadores predicen

$$\hat{y}_1(x) = 1, \qquad \hat{y}_2(x) = 1, \qquad \hat{y}_3(x) = 0,$$

entonces el ensamble predice $\hat{y}(x) = 1$.

¿Por qué puede funcionar la votación?

Consideremos M clasificadores independientes, cada uno con probabilidad $p > 0{,}5$ de acertar.
Si M es impar, la probabilidad de que la mayoría acierte es

$$P(\text{mayoría correcta}) = \sum_{k=(M+1)/2}^{M} \binom{M}{k} p^k (1-p)^{M-k}.$$

Ejemplo

Si cada clasificador acierta con probabilidad $p = 0{,}51$ y tenemos $M = 1001$ predictores independientes, la probabilidad de que la mayoría acierte es aproximadamente 0,74.

Limitación importante

En la práctica, los clasificadores no son independientes: al entrenarse con datos similares pueden cometer los mismos errores.
Por ello, **la diversidad entre modelos es esencial**.

Hard Voting con Scikit-Learn

Utilizaremos el conjunto de datos make_moons y tres clasificadores diferentes.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
    =42)

voting_clf = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(random_state=42)),
        ("rf", RandomForestClassifier(random_state=42)),
        ("svc", SVC(random_state=42))
    ],
    voting="hard"
)
voting_clf.fit(X_train, y_train)
```

Comparación de clasificadores

Evaluamos los clasificadores individuales y el ensamble sobre el conjunto de prueba.

```
for name, clf in voting_clf.named_estimators_.items():
    print(name, clf.score(X_test, y_test))

print("Ensemble:", voting_clf.score(X_test, y_test))
```

Se obtienen los siguientes resultados:

Clasificador	**Accuracy**
Logistic Regression	0.864
Random Forest	0.896
SVC	0.896
Voting Classifier	**0.912**

Observación

El ensamble supera a los tres clasificadores individuales, aunque utiliza sus predicciones para tomar la decisión.

Soft Voting: votación probabilística

En lugar de votar directamente por una clase, cada modelo estima las probabilidades de pertenencia a cada clase.

Soft Voting

Para M clasificadores, calculamos

$$\overline{P}(y = k \mid x) = \frac{1}{M} \sum_{j=1}^{M} P_j(y = k \mid x).$$

La predicción final es

$$\boxed{\hat{y}(x) = \arg\max_k \overline{P}(y = k \mid x).}$$

- **Hard voting:** cada modelo aporta una clase.
- **Soft voting:** cada modelo aporta probabilidades.
- Soft voting considera la confianza de los modelos.

Soft Voting en Scikit-Learn

Para utilizar soft voting, todos los clasificadores deben permitir la estimación de probabilidades.

```
voting_clf.set_params(
    voting="soft",
    svc__probability=True
)

voting_clf.fit(X_train, y_train)

print(voting_clf.score(X_test, y_test))
```

El resultado es: Accuracy $=$ 0,920.

- El SVC requiere `probability=True`.
- Este ajuste aumenta el costo de entrenamiento.
- Las probabilidades se promedian para elegir la clase final.

Comparación

Hard voting: 91,2 %.
Soft voting: 92,0 %.

Bagging y Pasting

Otra forma de generar diversidad consiste en entrenar el mismo algoritmo sobre diferentes muestras aleatorias.

Bagging: Bootstrap Aggregating

Cada predictor se entrena con una muestra aleatoria **con reemplazo** del conjunto de entrenamiento.
Una observación puede aparecer varias veces en la muestra de un mismo predictor.

Pasting

Cada predictor se entrena con una muestra aleatoria **sin reemplazo**.
Una observación aparece como máximo una vez en la muestra de cada predictor.

En ambos casos, una observación puede ser utilizada para entrenar distintos predictores.

Agregación de modelos en Bagging

Supongamos que entrenamos M modelos $f_1, \ldots, f_M$ sobre diferentes muestras.

Regresión

La predicción suele calcularse mediante el promedio:

$$\hat{f}(x) = \frac{1}{M} \sum_{j=1}^{M} f_j(x).$$

Clasificación

Una opción es elegir la clase más frecuente:

$$\hat{y}(x) = \text{mode}\Big(\hat{y}_1(x), \ldots, \hat{y}_M(x)\Big).$$

La agregación permite reducir la variabilidad de las predicciones individuales. En particular, bagging suele ser efectivo con modelos de **bajo sesgo y alta varianza**, como los árboles de decisión profundos.

¿Por qué Bagging reduce la varianza?

Consideremos M predictores $f_1, \ldots, f_M$, con errores independientes y la misma varianza σ^2, evaluados en un punto fijo x.
En Bagging para regresión, la predicción final es el promedio:

$$\bar{f}(x) = \frac{1}{M} \sum_{j=1}^{M} f_j(x).$$

Dos propiedades de la varianza

1. Para una constante a:

$$\mathrm{Var}(aX) = a^2 \, \mathrm{Var}(X).$$

2. Para variables aleatorias independientes:

$$\mathrm{Var}\left(\sum_{j=1}^{M} X_j \right) = \sum_{j=1}^{M} \mathrm{Var}(X_j).$$

Reducción de varianza: demostración

Partimos de

$$\mathrm{Var}(\bar{f}(x)) = \mathrm{Var}\left(\frac{1}{M}\sum_{j=1}^{M} f_j(x)\right).$$

Aplicando las propiedades de la varianza:

$$\mathrm{Var}(\bar{f}(x)) = \frac{1}{M^2}\,\mathrm{Var}\left(\sum_{j=1}^{M} f_j(x)\right) = \frac{1}{M^2}\sum_{j=1}^{M}\mathrm{Var}(f_j(x)) = \frac{1}{M^2}\underbrace{(\sigma^2 + \cdots + \sigma^2)}_{M \text{ términos}}.$$

Por tanto,

$$\boxed{\mathrm{Var}(\bar{f}(x)) = \frac{\sigma^2}{M}.}$$

Interpretación

Al promediar M predictores independientes con igual varianza, la varianza del ensamble disminuye en un factor de M.
Por ejemplo, con $M = 100$, la varianza es 100 veces menor que la de un predictor individual.

Bagging: predictores correlacionados

En la práctica, los árboles de Bagging no son completamente independientes, pues se entrenan con muestras del mismo conjunto de datos.

Varianza de una suma: caso general

$$\mathrm{Var}\left(\sum_{j=1}^{M} X_j\right) = \sum_{j=1}^{M} \mathrm{Var}(X_j) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j).$$

Supongamos que todos los predictores tienen varianza σ^2 y correlación común ρ. Entonces,

$$\mathrm{Cov}(f_i(x), f_j(x)) = \rho\sigma^2, \qquad i \neq j.$$

Aplicando la fórmula al promedio (y notando que hay $\frac{M(M-1)}{2}$ pares distintos):

$$\mathrm{Var}(\bar{f}(x)) = \frac{1}{M^2}\left[M\sigma^2 + M(M-1)\rho\sigma^2\right] = \frac{\sigma^2}{M}\left[1 + (M-1)\rho\right].$$

Finalmente,

$$\boxed{\mathrm{Var}(\bar{f}(x)) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right).}$$

Bagging: efecto de la correlación

La varianza del ensamble está dada por

$$\boxed{\text{Var}(\bar{f}(x)) = \sigma^2 \left(\rho + \frac{1-\rho}{M} \right).}$$

1. **Predictores independientes ($\rho = 0$):**

$$\text{Var}(\bar{f}(x)) = \frac{\sigma^2}{M} \xrightarrow[M \to \infty]{} 0.$$

   La varianza disminuye conforme aumenta M.

2. **Predictores perfectamente correlacionados ($\rho = 1$):**

$$\text{Var}(\bar{f}(x)) = \sigma^2.$$

   Promediar no produce ninguna reducción de varianza.

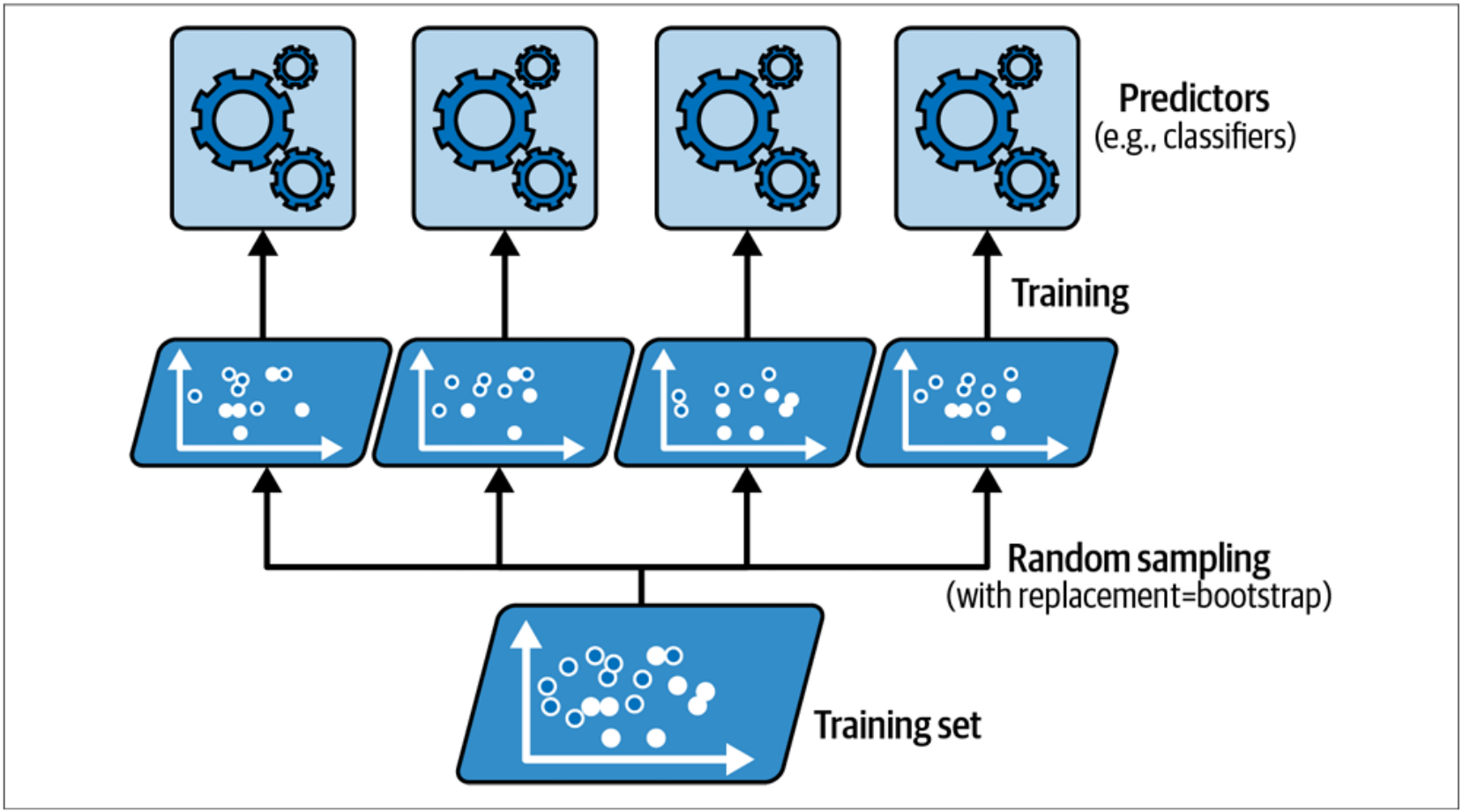
3. **Predictores parcialmente correlacionados ($0 < \rho < 1$):**

$$\lim_{M \to \infty} \text{Var}(\bar{f}(x)) = \rho \sigma^2.$$

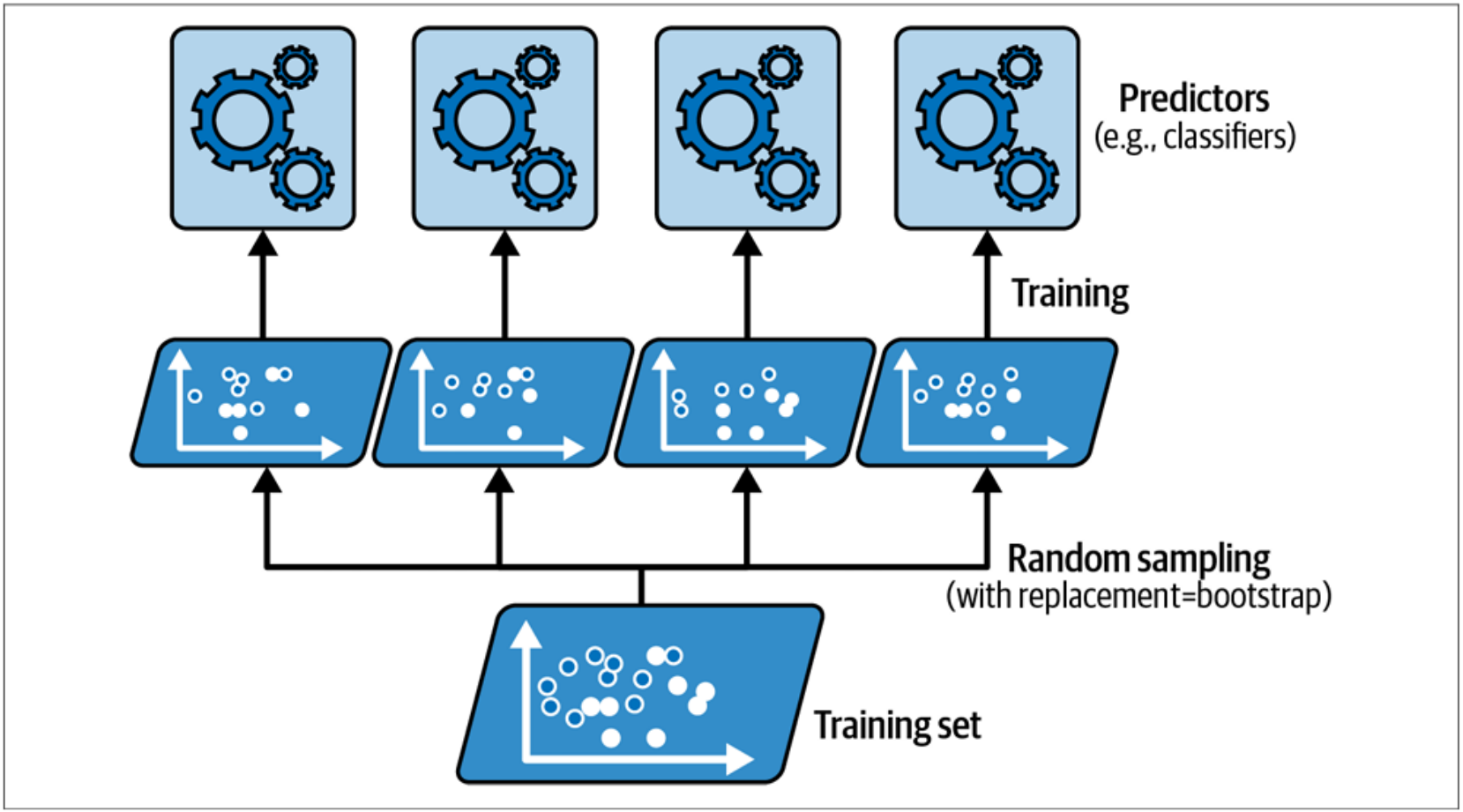
   Existe un límite en la reducción de varianza.

Conclusión: Bagging reduce la varianza mediante el promedio, pero su efectividad depende de la correlación entre los predictores. Cuanto menor sea ρ, mayor podrá ser la reducción de varianza.

Bagging y pasting escalan bien:



Bagging y pasting escalan bien:



¡Los clasificadores pueden entrenarse en paralelo!

Bagging con árboles de decisión

Scikit-Learn proporciona `BaggingClassifier` y `BaggingRegressor`.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

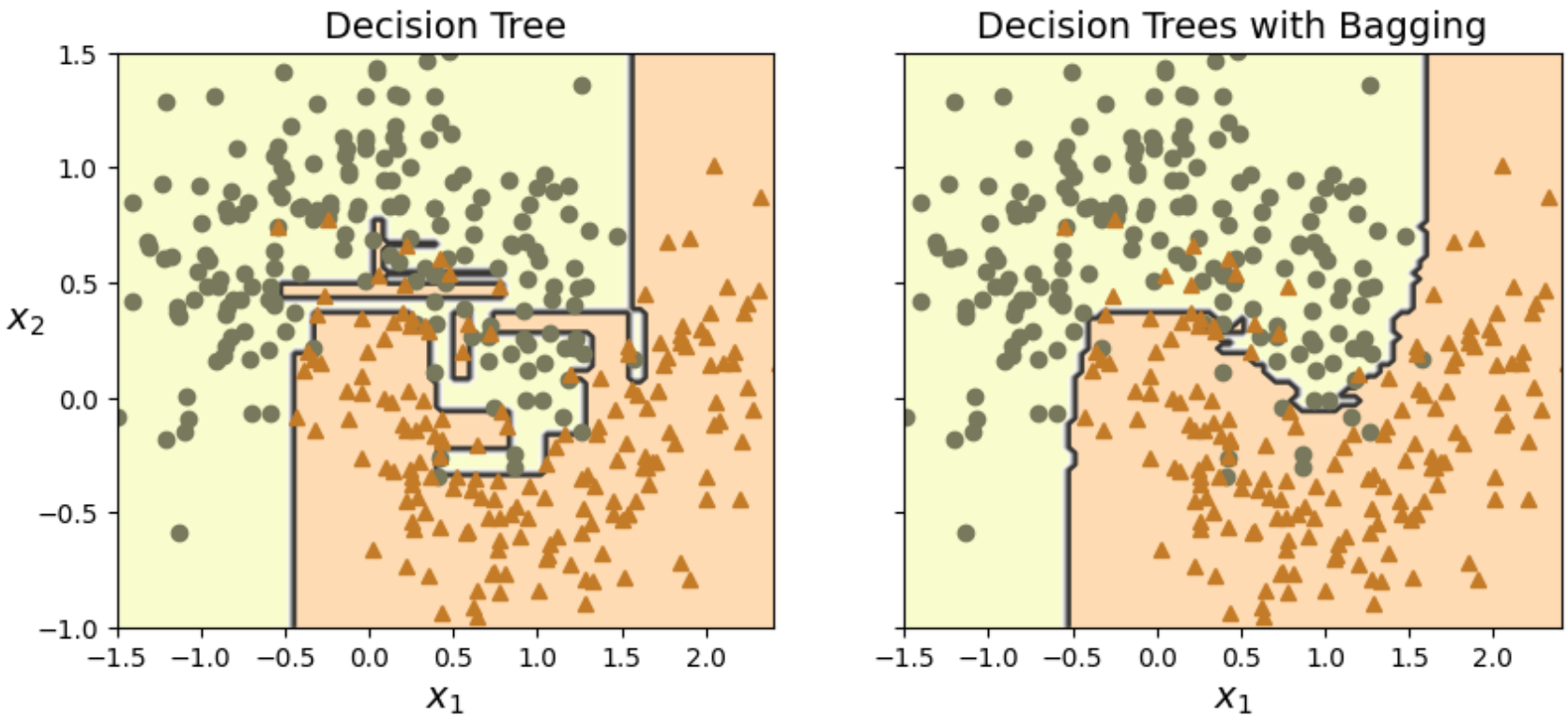
bag_clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=500,
    max_samples=100,
    bootstrap=True,
    n_jobs=-1,
    random_state=42
)

bag_clf.fit(X_train, y_train)
print(bag_clf.score(X_test, y_test))
```

- `n_estimators=500`: número de árboles.
- `max_samples=100`: tamaño de cada muestra.
- `bootstrap=True`: muestreo con reemplazo.
- `n_jobs=-1`: utiliza todos los núcleos disponibles.

Para utilizar pasting, basta cambiar `bootstrap=False`.

Fronteras de decisión: árbol vs bagging



Out-of-Bag Evaluation

En bagging, cada predictor recibe una muestra bootstrap del conjunto de entrenamiento.
Al utilizar m extracciones con reemplazo, la probabilidad de que una observación no sea seleccionada es

$$P(\text{no seleccionada}) = \left(1 - \frac{1}{m}\right)^m.$$

Cuando m es grande,[1]

$$\left(1 - \frac{1}{m}\right)^m \longrightarrow e^{-1} \approx 0{,}368.$$

Consecuencia: En promedio, aproximadamente el 63,2 % de las observaciones distintas aparecen en cada muestra bootstrap. El 36,8 % restante queda **fuera de la muestra** (*out-of-bag*, OOB).
Estos datos pueden utilizarse para evaluar los modelos sin reservar un conjunto de validación adicional.

[1] $\lim_{n\to\infty}\left(1 + \frac{a}{n}\right)^n = e^a$.

Evaluación Out-of-Bag en Scikit-Learn

```
bag_clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=500,
    oob_score=True,
    n_jobs=-1,
    random_state=42
)

bag_clf.fit(X_train, y_train)

print("OOB:", bag_clf.oob_score_)
print("Test:", bag_clf.score(X_test, y_test))
```

Resultados del ejemplo:

$$\text{OOB Accuracy} = 0{,}896, \qquad \text{Test Accuracy} = 0{,}920.$$

- Cada observación es evaluada mediante los modelos que no la utilizaron durante su entrenamiento.
- `oob_score_` contiene la evaluación OOB.

Random Patches y Random Subspaces

El muestreo no tiene que limitarse a las observaciones: también podemos seleccionar características aleatoriamente.

Random Patches

Cada predictor se entrena con:

- Un subconjunto aleatorio de observaciones.
- Un subconjunto aleatorio de características.

Random Subspaces

Se utilizan todas las observaciones de entrenamiento, pero solamente un subconjunto aleatorio de características.

- `max_features`: cantidad de características.
- `bootstrap_features`: permite seleccionar características con reemplazo.

Compromiso

El muestreo de características incrementa la diversidad y puede reducir la varianza, pero también aumentar el sesgo.

Random Forests: bosques aleatorios

Un **Random Forest** es un ensamble de árboles de decisión que incorpora aleatoriedad durante el entrenamiento.

1. Se generan diferentes muestras del conjunto de entrenamiento, normalmente mediante bootstrap.
2. Se entrena un árbol de decisión sobre cada muestra.
3. En cada nodo se considera solamente un **subconjunto aleatorio de características** para buscar la mejor división.
4. Las predicciones de los árboles se agregan.

Diferencia respecto de Bagging

En un Random Forest, además de variar las muestras, se introduce aleatoriedad en la selección de características candidatas en cada nodo.

Esto reduce la correlación entre los árboles y favorece la reducción de varianza.

Random Forests en Scikit-Learn

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
    n_jobs=-1, random_state=42)

rnd_clf.fit(X_train, y_train)

y_pred = rnd_clf.predict(X_test)
```

Parámetros relevantes:

- `n_estimators`: número de árboles.
- `max_leaf_nodes`: máximo de hojas por árbol.
- `max_features`: número de características candidatas por división.
- `bootstrap`: controla el muestreo de observaciones.

Valor predeterminado

En clasificación, `max_features="sqrt"`: se consideran aproximadamente $\sqrt{n}$ características en cada nodo, donde n es el número total de características.

Extra-Trees: árboles extremadamente aleatorios

Los Random Forests introducen aleatoriedad al seleccionar las características candidatas.
Los **Extra-Trees** añaden otra fuente de aleatoriedad:

- Seleccionan un subconjunto aleatorio de características.
- Generan umbrales aleatorios para esas características.
- Eligen la mejor división entre los umbrales propuestos.

Consecuencias

- Mayor diversidad entre árboles.
- Posible aumento del sesgo.
- Posible reducción de varianza.
- Entrenamiento frecuentemente más rápido.

En Scikit-Learn se implementan mediante `ExtraTreesClassifier` y `ExtraTreesRegressor`.
A diferencia de Random Forest, en Extra-Trees `bootstrap=False` por defecto.

Comparación: Random Forest y Extra-Trees

```
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=300, random_state=42
    ),
    "Extra-Trees": ExtraTreesClassifier(
        n_estimators=300, random_state=42
    )
}

for name, model in models.items():
    model.fit(X_train, y_train)
    print(name, model.score(X_test, y_test))
```

¿Es siempre mejor Extra-Trees que Random Forest?
No. La aleatoriedad adicional puede mejorar la generalización, pero también provocar subajuste.
Es necesario comparar ambos métodos mediante validación.

Importancia de las características y Boosting

Próxima clase ...

¡Gracias!